

Oblak - platforma za izvršavanje korisničkog koda

June 2, 2026

1 Uvod

Oblak je platforma za izvršavanje korisničkog Python koda na serveru, po ideji slična AWS Lambda i Google Cloud funkcijama. Korisnik preko konzolne aplikacije (CDK CLI) pošalje svoj kod na server; server ga proveri, pripremi i napravi URL. Pozivom tog URL-a (npr. običnim cURL-om) kod se pokrene i vrati rezultat.

Pošto je kod koji se izvršava potencijalno opasan - ne znamo šta je korisnik poslao - ceo fokus projekta je na bezbednosti: kako da taj kod izvršimo a da ne ugrozimo host ni druge korisnike. Za izolaciju koristim Firecracker, koji za svaki poziv pravi zasebnu mikro virtuelnu mašinu.

2 Funkcionalnosti

Implementirane su sve tražene funkcionalnosti:

- **Autentikacija CDK CLI ka serveru** - preko API ključa (Bearer token).
- **Prenos koda na oblak** - `deploy` šalje `.py` fajl, po potrebi i `requirements.txt`.
- **Analiza prenetog koda** - antivirus (ClamAV), statička analiza (Bandit i sopstveni AST skener) i opcionalna LLM analiza.
- **Priprema koda za izvršavanje** - povlačenje zavisnosti na bezbedan način.
- **Kreiranje URL-a** - po `deploy`-u se vraća `invoke` URL sa tokenom.
- **Pokretanje koda na zahtev** - izvršavanje u Firecracker microVM-u.

3 Arhitektura i tok

Sistem ima tri dela: **CDK CLI** (`cli/cdk.py`) za autentikaciju, upload, pregled statusa i poziv; **server** (FastAPI, `server/`) koji prima zahteve, čuva kod u SQLite bazi i pokreće verifikaciju; i **runtime** (`guest/` + `server/orchestrator/`) koji čine guest agent unutar VM-a i orchestrator na hostu.

Tok: CLI se autentikuje i pošalje kod. Server ga sačuva i u pozadini pokrene verifikaciju. Ako kod prođe, status postaje `ready`. Poziv ide na `/invoke/{id}?token=...`; server učitava kod, orchestrator digne novu VM, agent u njoj izvrši kod i vrati izlaz preko diska, a server ga prosledi pozivaocu.

4 Bezbednosni zahtevi

Pre implementacije definisao sam šta sistem mora da zadovolji:

1. Nepouzdan kod ne sme da kompromituje host ni da vidi/menja podatke drugih korisnika.

2. Izvršavanje ne sme da ima izlaz na mrežu (sprečiti exfiltraciju i vezu ka napolju).
3. Sve operacije nad funkcijama traže autentikaciju; korisnik vidi samo svoje funkcije.
4. Tajne (API ključevi, tokeni) se ne čuvaju u čistom tekstu.
5. Bezbednosno bitne radnje moraju biti zabeležene i otporne na naknadnu izmenu.
6. Mora postojati ograničenje resursa (CPU, memorija, vreme, procesi).
7. Povlačenje zavisnosti ne sme da izvrši nepouzdan kod.

5 Analiza izolacije izvršavanja u VM

Postavka traži da se posebno analizira izvršavanje koda u virtuelnoj mašini, pre svega izolovanost procesa. Polazna pretpostavka je da je korisnički kod neprijateljski i da će pokušati da pobegne ili da naškodi.

Glavni razlog za microVM umesto kontejnera je što VM ima *sopstveni kernel* i ne deli ga sa hostom. Kod kontejnera svi procesi dele host kernel, pa jedna ranjivost kernela vodi do bekstva; kod Firecracker-a granica je KVM (hardverska virtuelizacija), što je mnogo jača izolacija. Na to sam dodao više slojeva:

- **Bez mreže** - VM-u se ne dodeljuje nijedan mrežni uređaj, pa kod nema kako da otvori vezu napolje. Time je zahtev Z2 ispunjen na nivou arhitekture, ne politike.
- **Neprivilegovan proces** - korisnički kod se ne pokreće kao root nego kao **nobody**; čak i unutar VM-a nema administratorske privilegije.
- **Read-only root disk** - root je montiran samo za čitanje i deljen je među VM-ovima, pa kod ne može da ga izmeni ni da ostavi trag za naredni poziv. Pisanje ide samo u **tmpfs** /**tmp** (u RAM-u), koji nestaje gašenjem VM-a.
- **Ograničenje resursa** - CPU sekunde, broj procesa i veličina fajla ograničeni su u agentu, plus 1 vCPU i fiksna količina RAM-a (Firecracker), plus „wall-clock” timeout na hostu koji ubija VM ako pređe rok.
- **Jedna VM po pozivu** - nema deljenja stanja između poziva ni korisnika.

Komunikacija host-VM ide preko diska: host upiše ulaz u mali ext4 disk, agent ga pročitava, izvrši kod i upiše izlaz, pa se VM ugasi. Ovak pristup ne zahteva mrežu i koristi samo virtio-blk uređaj.

Ostatak rizika: i dalje postoji teorijska mogućnost eksploita guest kernela ili samog Firecracker procesa. Taj rizik se dodatno smanjuje pokretanjem Firecracker-a pod **jailer**-om (chroot, namespaces, cgroups, drop privilegija), što je implementirano ali podrazumevano isključeno (videti otvorene stavke).

6 Model pretnji

Najvredniji resurs je host i njegov kernel - kompromis tu znači gubitak svega. Slede kod i podaci drugih korisnika, pa tajne (ključevi i tokeni), pa revizioni dnevnik.

Granice poverenja:

- Mreža → server: brani je API ključ, odnosno invoke token.
- Server → VM: glavna granica; sve unutar VM-a je neprijateljsko.
- Build host → VM: skidanje zavisnosti koristi mrežu hosta, a VM ostaje bez nje.

Akteri: CLI korisnik (autentikovano, polupovereno), invoke pozivalac (anoniman, ima samo token), i korisnički kod (potpuno nepovereno).

7 STRIDE analiza

Pretnje sam sistematizovao po STRIDE metodologiji (Tabela 1).

Kategorija	Pretnja	Mehanizam ublažavanja
Spoofing	predstavljanje kao drugi korisnik / poziv tuđe funkcije	API ključevi (256-bit, čuva se samo HMAC heš); zaseban invoke token po funkciji
Tampering	izmena koda ili baze, SQL injection, path traversal	parametrizovani SQL; kod pod uuid folderom sa fiksnim imenima fajlova; root disk read-only
Repudiation	poricanje da se radnja desila	append-only audit log sa heš-lancem
Information Disclosure	čitanje tuđih podataka, curenje tajni, exfiltracija	upiti ograničeni na vlasnika; tajne heširane; VM bez mreže; sveža VM po pozivu
Denial of Service	beskonačne petlje, fork bombe, trošenje memorije	wall-clock timeout, 1 vCPU i ograničena RAM, limiti procesa u agentu
Elevation of Privilege	bekstvo iz sandbox-a, root u guestu	Firecracker microVM (zaseban kernel); kod kao nobody; opciono jailer

Tabela 1: STRIDE analiza pretnji i mehanizmi ublažavanja.

8 Mehanizmi ublažavanja

Autentikacija. API ključ je nasumičan token od 256 bita; u bazi se čuva samo HMAC-SHA256 heš sa server-side „pepper”-om. Nisam koristio argon2 jer spori hešovi služe protiv lozinki male entropije, a token od 256 bita se ne može pogoditi. Invoke token je odvojen od API ključa, jer pozivalac (cURL) ne treba da poseduje korisnički ključ.

Revizija (audit). Sve bitne radnje (deploy, verifikacija, svaki poziv, start/stop VM-a) upisuju se u tabelu `audit_log` koja se nikad ne menja niti briše. Svaki zapis sadrži heš prethodnog, pa naknadna izmena bilo kog reda prekida lanac i funkcija `verify_chain()` to otkriva. Ovo sam i testirao - ručnom izmenom jednog reda provera je prijavila grešku tačno na tom mestu.

Verifikacija koda. Pre nego što postane pozivljiv, kod prolazi kroz antivirus (ClamAV), Bandit i sopstveni AST skener. Pošto je ovo platforma za izvršavanje proizvoljnog koda, korisnik sme da koristi `eval` ili `subprocess`; verifikacija ih zato samo beleži (radi revizije), a ne blokira automatski. Tvrde blokade su za pravi virus i za obfuskaciju tipa `exec(base64.b64decode(...))`. Strogi režim (`OBLAK_VERIFIER_STRICT`) blokira i te slabije signale.

Priprema zavisnosti. `pip install` sdist paketa pokreće `setup.py`, što je izvršavanje nepoznatog koda. Zato skidam samo wheel-ove (`--only-binary=:all:`), instaliram ih offline u zaseban folder i pakujem u disk koji se montira read-only u VM bez mreže. Tako se build kod nepoznatog paketa nikad ne izvrši.

9 Statička analiza sopstvenog koda

Postavka traži i skeniranje samog softvera alatom za statičku analizu. Pokrenuo sam Bandit nad celim kodom (`scripts/selfscan.sh`). Nije bilo HIGH nalaza. Nekoliko Low/Medium nalaza postoji, ali su očekivani: agent koristi `subprocess` jer baš pokreće korisnički kod, i koristi `/tmp` jer je to tmpfs u efemernom VM-u. To su svesne odluke, ne propusti.

10 Testiranje

Napravljeni su testni primeri u `tests/`, benigni i maliciozni.

Benigni: osnovni ispis i računica, čitanje sa standardnog ulaza, i primer koji koristi eksternu zavisnost (`requests`) radi provere pripreme zavisnosti.

Maliciozni i odbrana koja ih hvata:

- obfuskacija (`exec(base64...)`) - odbijena verifikacijom u strogom režimu;
- reverse shell - konekcija pada jer VM nema mrežu;
- fork bomba i beskonačna petlja - timeout gasi VM (`timed_out=true`);
- pokušaj čitanja `/etc/shadow` i pisanja na root - pada zbog `nobody` i read-only diska.

Svaki primer je pokrenut kroz platformu i ponašao se kako se očekuje.

11 Mogucnosti unapredjenja

- Server radi preko HTTP-a na localhost-u; u pravoj upotrebi neophodan je TLS.
- Jailer je implementiran ali podrazumevano isključen (na WSL2 cgroups traže doterivanje); glavnu izolaciju ionako daje VM.
- Svaki poziv je „hladan start”; warm pool VM-ova bi smanjio latenciju.
- Nema rate-limitinga ni kvota po korisniku.
- Invoke token ide u query string, što može završiti u logovima; bolje kroz header.